Capture the Flag 4: The Student List

Goal: Build a scrollable student directory using Dart Lists and `ListView.builder`, replacing manually duplicated widgets with dynamically generated UI.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

- Research what a Dart/Flutter feature does
- Understand syntax
- Explain an error (not fix your error)
- Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

🚩 Flag 0 — Create Your Feature Branch

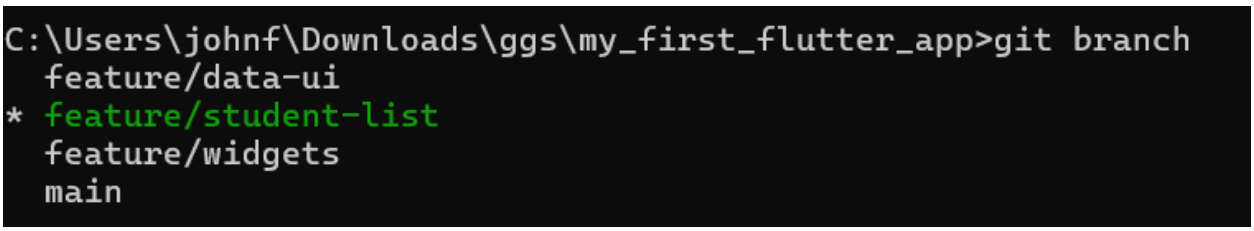
Objective: Create a feature branch before building your student list.

Create:

`feature/student-list`

Screenshot:

- Terminal showing your current branch



🚩 Flag 1 — Build Your Student Data

Objective: Store multiple students inside a Dart `List`.

Research what a Dart `List` is and why it is useful when handling multiple pieces of similar data.

Instead of creating one student at a time with separate variables, create a collection that stores at least five students.

Each student should contain information such as:

- Image
- Name
- Course
- Year Level
- Age
- Hobby

Your data should exist separately from your UI widgets.

Screenshot:

Code:

- Your student list
- At least five student entries

```dart
class Student {
  final String image;
  final String name;
  final String course;
  final String yearLevel;
  final int age;
  final String hobby;

  const Student({
    required this.image,
    required this.name,
    required this.course,
    required this.yearLevel,
    required this.age,
    required this.hobby,
  });
}
```

```dart
const List<Student> students = [
  Student(
    image: "assets/profile2.jpg",
    name: "Ivan Villareal",
    course: "BSIT",
    yearLevel: "3rd Year",
    age: 24,
    hobby: "Photography, Videography, & Video Editing",
  ),

  Student(
    image: "assets/Profile1.jpg",
    name: "Mannes Artajo",
    course: "BSIT",
    yearLevel: "2nd Year",
    age: 21,
    hobby: "Reading"
```

```
47      Student(
48        image: "assets/profile3.jpg",
49        name: "Wilken Montebon",
50        course: "BSIT",
51        yearLevel: "1st Year",
52        age: 20,
53        hobby: "Basketball",
54      ),
55
56      Student(
57        image: "assets/profile4.png",
58        name: "John Kevin Villacorte",
59        course: "BSIT",
60        yearLevel: "4th Year",
61        age: 23,
62        hobby: "Drawing",
63      ),
```

```
65      Student(
66        image: "assets/profile5.jpg",
67        name: "Keith Francheska Lopez",
68        course: "BSIT",
69        yearLevel: "3rd Year",
70        age: 22,
71        hobby: "Gaming",
72      ),
73    ];
```

🚩 Flag 2 — The Copy-Paste Trap

You already know how to create a Student Card.

You created five student cards manually.

You duplicated the same card design until all five students appear.

Stop.

Imagine having:

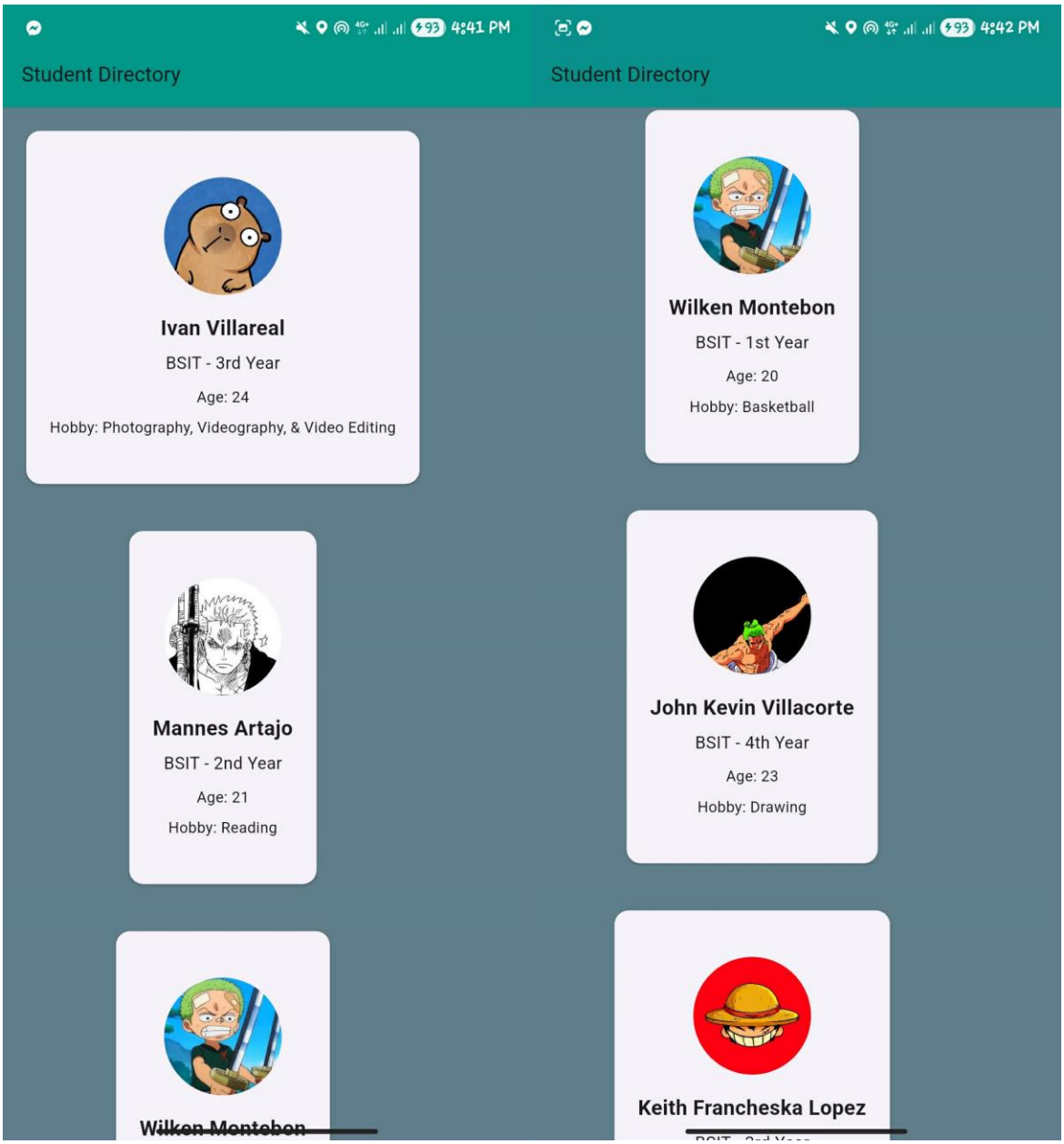
- 50 students.
- 500 students.
- 5,000 students.

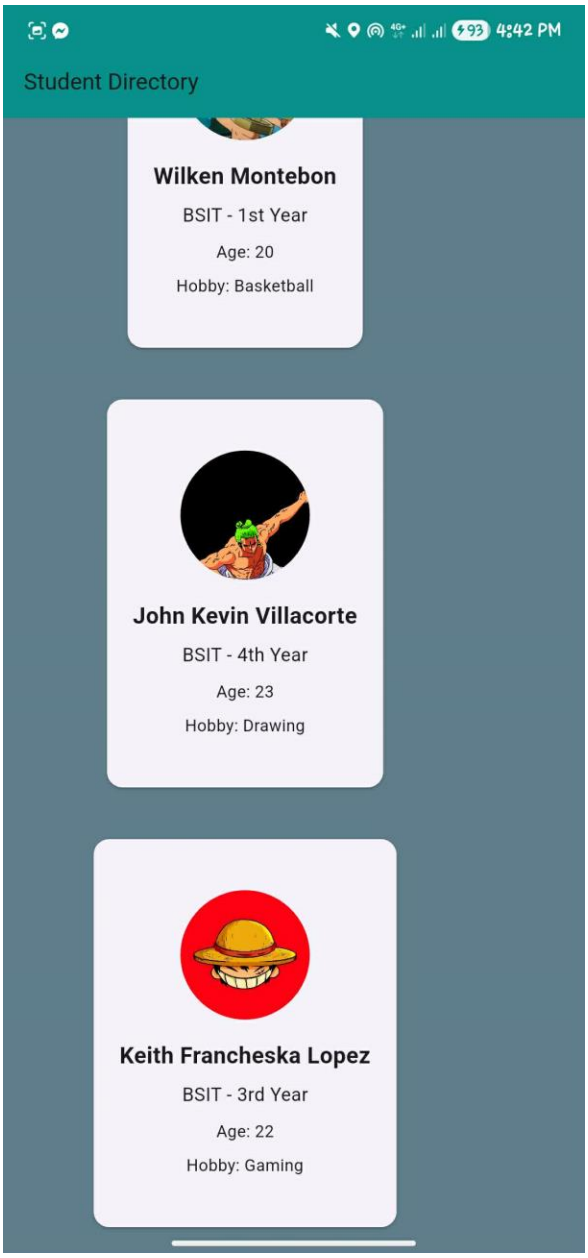
Would copying the same widget over and over still be practical?

This flag is intentionally making you feel the problem before solving it.

Screenshot:

- Your five manually created student cards (do not change your past code yet)



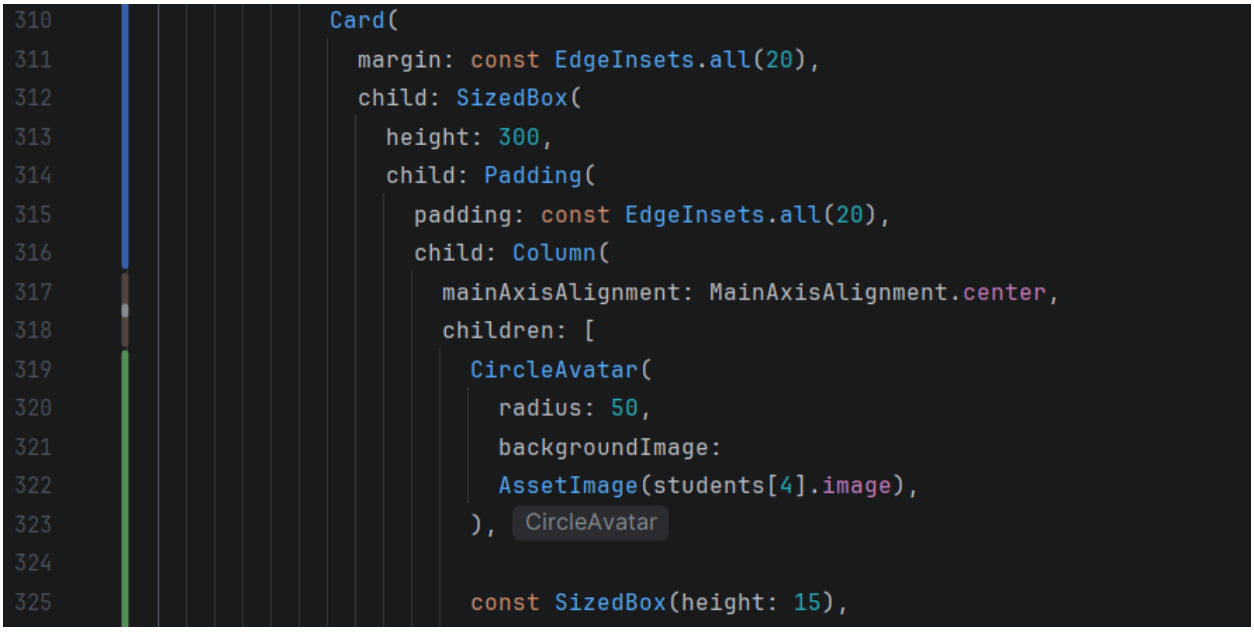

4:42 PM
Student Directory
Wilken Montebon
BSIT - 1st Year
Age: 20
Hobby: Basketball
John Kevin Villacorte
BSIT - 4th Year
Age: 23
Hobby: Drawing
Keith Francheska Lopez
BSIT - 3rd Year
Age: 22
Hobby: Gaming

```
94                Card(
95                  margin: const EdgeInsets.all(20),
96                  child: SizedBox(
97                    height: 300,
98                    child: Padding(
99                      padding: const EdgeInsets.all(20),
100                     child: Column(
101                       mainAxisAlignment: MainAxisAlignment.center,
102                       children: [
103                         CircleAvatar(
104                           radius: 50,
105                           backgroundImage:
106                           AssetImage(students[0].image),
107                         ), CircleAvatar
```

```
148               Card(
149                 margin: const EdgeInsets.all(20),
150                 child: SizedBox(
151                   height: 300,
152                   child: Padding(
153                     padding: const EdgeInsets.all(20),
154                     child: Column(
155                       mainAxisAlignment: MainAxisAlignment.center,
156                       children: [
157                         CircleAvatar(
158                           radius: 50,
159                           backgroundImage:
160                           AssetImage(students[1].image),
161                         ), CircleAvatar
162
```

```
203                 margin: const EdgeInsets.all(20),
204                 child: SizedBox(
205                   height: 300,
206                   child: Padding(
207                     padding: const EdgeInsets.all(20),
208                     child: Column(
209                       mainAxisAlignment: MainAxisAlignment.center,
210                       children: [
211                         CircleAvatar(
212                           radius: 50,
213                           backgroundImage:
214                           AssetImage(students[2].image),
215                         ), CircleAvatar
216
217                         const SizedBox(height: 15),
```

```
256                 Card(
257                   margin: const EdgeInsets.all(20),
258                   child: SizedBox(
259                     height: 300,
260                     child: Padding(
261                       padding: const EdgeInsets.all(20),
262                       child: Column(
263                         mainAxisAlignment: MainAxisAlignment.center,
264                         children: [
265                           CircleAvatar(
266                             radius: 50,
267                             backgroundImage:
268                             AssetImage(students[3].image),
269                           ), CircleAvatar
270
271                           const SizedBox(height: 15),
```



```
310                 Card(
311                   margin: const EdgeInsets.all(20),
312                   child: SizedBox(
313                     height: 300,
314                     child: Padding(
315                       padding: const EdgeInsets.all(20),
316                       child: Column(
317                         mainAxisAlignment: MainAxisAlignment.center,
318                         children: [
319                           CircleAvatar(
320                             radius: 50,
321                             backgroundImage:
322                             AssetImage(students[4].image),
323                           ), CircleAvatar
324
325                           const SizedBox(height: 15),
```

🚩 Flag 3 — Discover `ListView`

Objective: Make your student directory scrollable using `ListView`.

Research Flutter's `ListView`.

Replace your previous layout with a scrolling layout so users can browse through all of your student cards naturally.

Your application should now allow continuous vertical scrolling.

Test

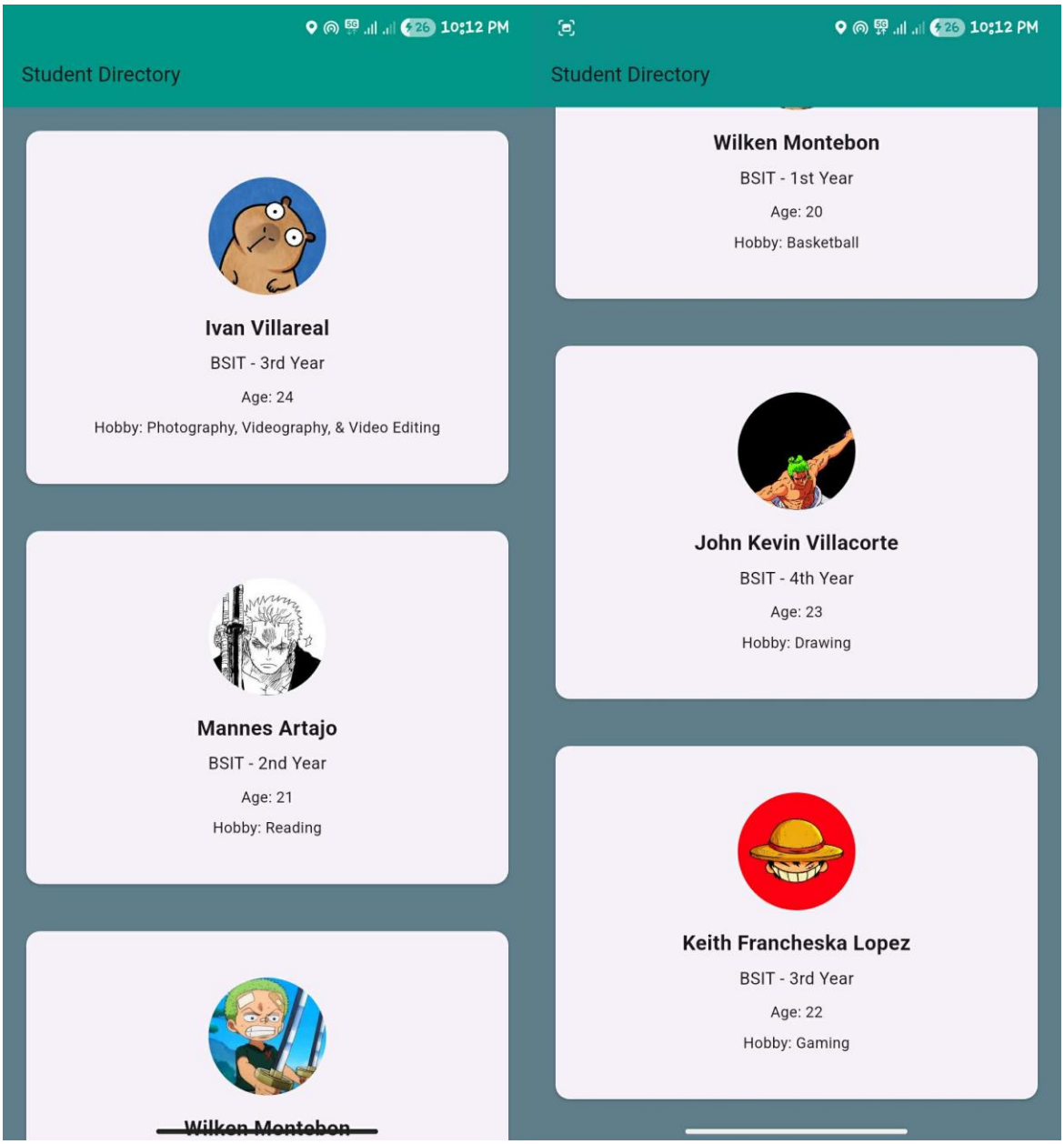
The user should be able to:

- Scroll from the first student
- Continue scrolling
- Reach the last student without resizing the screen

Screenshot:

- `ListView` in your code
- Top of the application
- Bottom portion showing later students

```
body: ListView(
  children: [
```



🚩 Flag 4 — Unlock `ListView.builder`

Objective: Generate student cards automatically using `ListView.builder`.

Research `ListView.builder`.

Instead of writing five separate Student Cards, generate them automatically from your student list.

Your UI should now create one card for each student in your list.

Test

Add one new student to your list.

Without creating another card manually, your application should display the new student automatically.

Screenshot:

- `ListView.builder` in your code
- Student list
- Newly generated student appearing in the app

```
103         body: ListView.builder(
104           itemCount: students.length,
105
106           itemBuilder: (context, index) {
107
108             final student = students[index];
109
110             return Card(
```

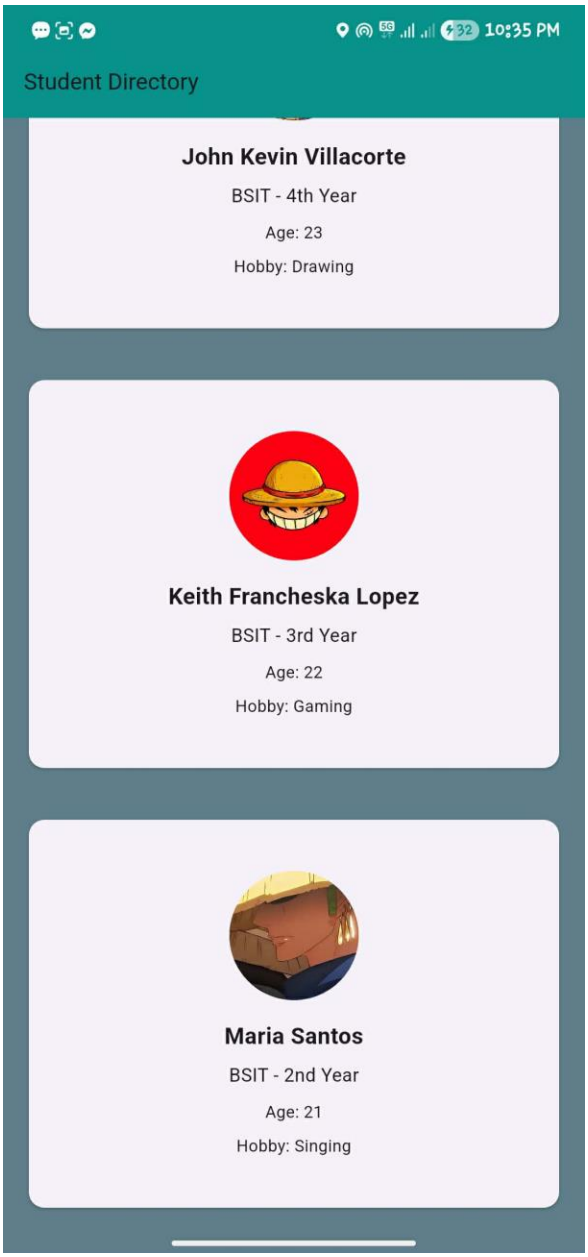
```
28  const List<Student> students = [
29    Student(
30      image: "assets/profile2.jpg",
31      name: "Ivan Villareal",
32      course: "BSIT",
33      yearLevel: "3rd Year",
34      age: 24,
35      hobby: "Photography, Videography, & Video Editing",
36    ),
37
38    Student(
39      image: "assets/Profile1.jpg",
40      name: "Mannes Artajo",
41      course: "BSIT",
42      yearLevel: "2nd Year",
43      age: 21,
44      hobby: "Reading",
45    ),
```

```
  Student(
    image: "assets/profile3.jpg",
    name: "Wilken Montebon",
    course: "BSIT",
    yearLevel: "1st Year",
    age: 20,
    hobby: "Basketball",
  ),

  Student(
    image: "assets/profile4.png",
    name: "John Kevin Villacorte",
    course: "BSIT",
    yearLevel: "4th Year",
    age: 23,
    hobby: "Drawing",
  ),
```

```
  Student(
    image: "assets/profile5.jpg",
    name: "Keith Francheska Lopez",
    course: "BSIT",
    yearLevel: "3rd Year",
    age: 22,
    hobby: "Gaming",
  ),

  // FLAG 4 - New Student
  Student(
    image: "assets/Profile6.jpg",
    name: "Maria Santos",
    course: "BSIT",
    yearLevel: "2nd Year",
    age: 21,
    hobby: "Singing",
  ),
];
```



🚩 Flag 5 — Every Student Deserves an ID

Objective: Expand your student records with additional information.

Real student directories contain more than just names.

Add at least three new fields to every student.

Examples:

- Student ID
- Email
- Favorite Subject

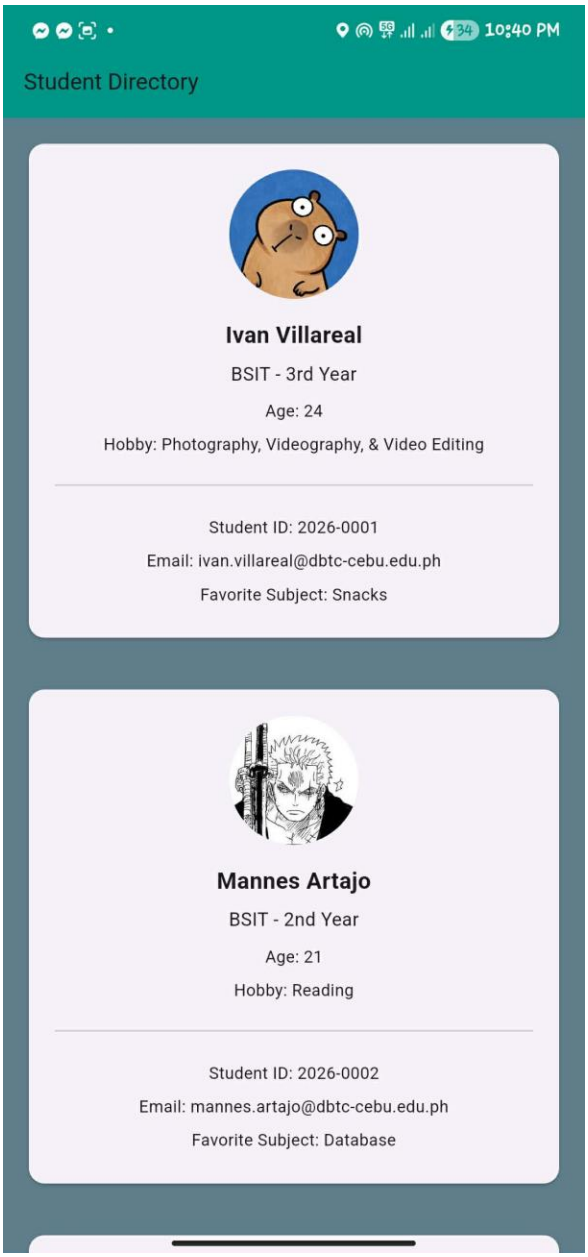
Update your Student Card so the new information appears consistently across every generated card.

Screenshot:

- Updated student data
- Student Card displaying the additional fields

```
Student(
  image: "assets/profile2.jpg",
  name: "Ivan Villareal",
  course: "BSIT",
  yearLevel: "3rd Year",
  age: 24,
  hobby: "Photography, Videography, & Video Editing",

  // FLAG 5
  studentId: "2026-0001",
  email: "ivan.villareal@dbtc-cebu.edu.ph",
  favoriteSubject: "Snacks",
),
```



🚩 Flag 6 — Empty Classroom

What happens when there are no students?

Objective: Handle an empty `List` gracefully.

Imagine your app just connected to a database.

The database returns:

[]

No students.

What should the user see?

Instead of displaying a broken or confusing screen, research how Flutter can display a different widget when a condition is met.

Create an empty state that clearly communicates that there are currently no students available.

Examples:

- "No students found."
- "Student list is empty."

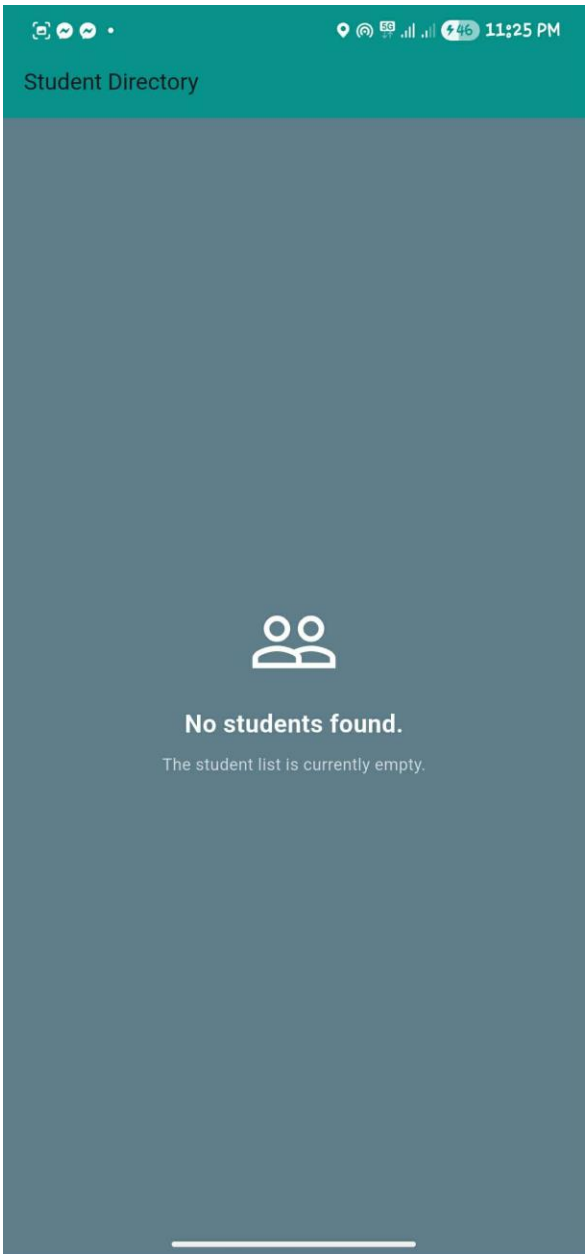
Test

- Display your student list normally.
- Then temporarily make your list empty.
- Your application should display the empty state instead of student cards.

Screenshot:

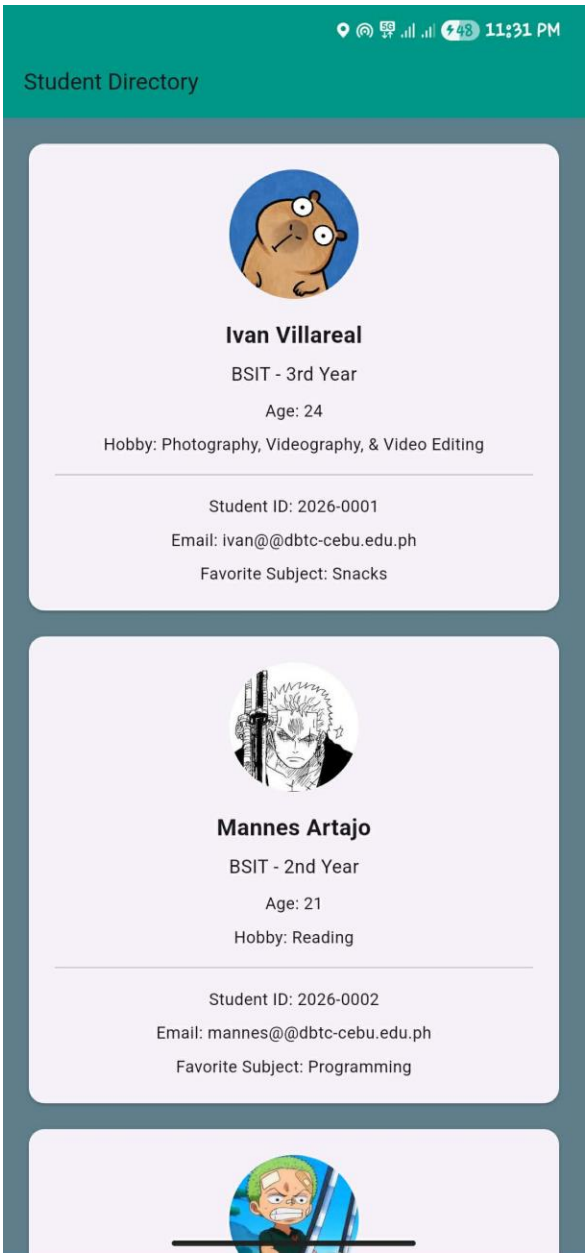
- Empty student list
- Empty-state UI
- Application still functioning correctly

```
35  const List<Student> students = [];
36
```



```
54          body: students.isEmpty
55              ? const Center(
56            child: Column(
57              mainAxisAlignment: MainAxisAlignment.center,
58              children: [
59                Icon(
60                  Icons.people_outline,
61                  color: Colors.white,
62                  size: 70,
63                ),  Icon
```

```
35  const List<Student> students = [
36    Student(
37      image: "assets/profile2.jpg",
38      name: "Ivan Villareal",
39      course: "BSIT",
40      yearLevel: "3rd Year",
41      age: 24,
42      hobby: "Photography, Videography, & Video Editing",
43      studentId: "2026-0001",
44      email: "ivan@@dbtc-cebu.edu.ph",
45      favoriteSubject: "Snacks",
46    ),
47
48    Student(
49      image: "assets/Profile1.jpg",
```



🚩 Flag 7 — Sort the Roll Call

Objective: Change the order of your student list.

Class lists are rarely displayed randomly.

Research how Dart can rearrange items inside a `List`.

Sort your students alphabetically by name.

Test

Before sorting:

- Maria
- John
- Aaron
- Sophia

After sorting:

- Aaron
- John
- Maria
- Sophia

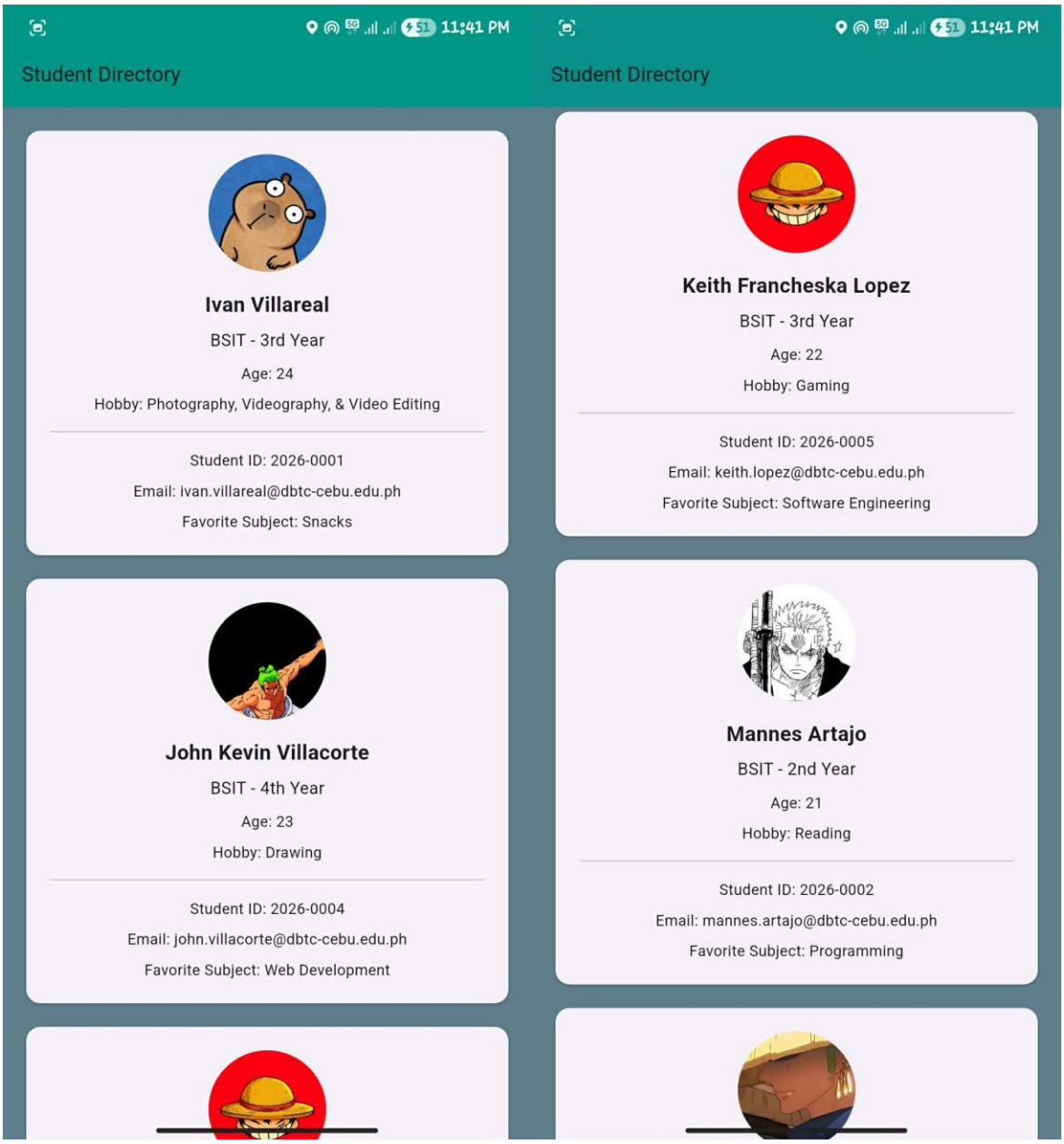
Your UI should immediately reflect the new order.

Screenshot:

- Student list before or after sorting
- Application displaying the sorted order

```
3   void main() {
4     // FLAG 7 - Sort students alphabetically before running the app
5     students.sort(
6           (a, b) => a.name.toLowerCase().compareTo(
7         b.name.toLowerCase(),
8       ),
9     );
```

```
43  List<Student> students = [
```



🏴 Flag 8 — Save Your Progress

Objective: Commit and push your completed Student List CTF.

Use an appropriate commit message following proper Git standards.

Branch:

`feature/student-list`

Screenshot:

- Successful commit
- Current branch
- Terminal showing the commit